System Architecture

AICTE Approval Workflow Portal

Version: 1.0 · **Status:** Prototype architecture · **Date:** 22 October 2026

1. Architectural approach

Use a modular web application with a shared backend and three role-specific views. Keep regulatory workflow rules, permissions, deadlines and audit events on the server. The frontend must not be the authority for security or workflow transitions.

2. Logical architecture

```
Browser (React + TypeScript)
■■■ Institution workspace
■■■ AICTE officer workspace
■■■ AICTE administrator workspace
■ HTTPS / authenticated API
▼
Application service layer
■■■ Authentication + authorisation
■■■ Institution/role scope
■■■ Application + workflow engine
■■■ Deficiency resolution
■■■ EVC scheduling
■■■ Compliance + document validity
■■■ Pre-submission document-check service
■■■ Notification + escalation service
■■■ Dashboard/report service
■■■ Audit service
■
■■■ PostgreSQL database
■■■ Private object storage
■■■ Scheduled jobs/queue
■■■ OCR/AI provider adapter
■■■ Optional authorised external-system adapters
```

3. Component responsibilities

Frontend

- Render the correct workspace from the authenticated session's verified permissions.
- Provide forms, tables, document upload, status timelines and reports.
- Show loading/error/permission states.
- Never trust a client-supplied role or organisation ID without server validation.

Authentication and authorisation

- Authenticate users.
- Resolve role, organisation scope and permitted actions.
- Enforce access on every server operation and storage access.
- Support institution, officer and administrator roles; add fine-grained subroles where necessary.

Workflow engine

- Read a versioned workflow definition.
- Validate current stage, prerequisites, actor permission and required fields.

- Execute permitted transition atomically.
- Write workflow event and audit event.
- Reject invalid transitions with a human-readable reason.

Document service

- Store files privately and link versions to application/obligation records.
- Record uploader, timestamp, file hash/metadata and scan state.
- Provide short-lived authorised download access.
- Preserve replacement history; do not silently overwrite the evidence record.

Pre-submission check service

- Validate file and extract text/OCR.
- Run configured deterministic checks.
- Call AI provider only through a server-side adapter when enabled.
- Save check version, findings, evidence/page references and limitations.
- Return explainable warnings; never make an official decision.

Deadline/notification service

- A scheduled job checks upcoming and overdue items.
- Create notifications and escalation records according to versioned policy.
- Use idempotency keys to prevent duplicate sends.
- Log delivery outcomes and retry failures safely.

Reporting service

- Aggregate data within the user's permitted scope.
- Calculate ageing using explicit timestamp and timezone rules.
- Avoid leaking cross-region or cross-institution records.

4. Principal data relationships

- One organisation can have many applications and obligations.
- One application has workflow events, documents, deficiencies and possibly EVC records.
- One deficiency can have multiple correction submissions and review events.
- One obligation can have multiple evidence documents and verification events.
- One document can have multiple versions and multiple pre-check runs.
- Notifications and escalations reference the relevant case, task or obligation.
- Audit logs reference actor and target entity, with safe metadata.
- Workflow definitions are versioned; each case records the version used.

5. Workflow execution

1. User requests an action.
2. Backend authenticates the session and loads permitted scope.
3. Backend loads the case and current workflow definition/version.

4. Backend checks allowed transition, prerequisites, due-date policy and required reason/evidence.
5. If valid, update the case and append workflow/audit events in a transaction.
6. Trigger notification/job creation after commit.
7. UI refreshes the case timeline and next-action summary.

Do not allow the browser to update `current_stage` directly. All transitions go through the workflow service.

6. Document pre-check execution

1. Upload to private staging/quarantine.
2. Validate file size/type and scan where infrastructure supports it.
3. Extract text/OCR and page-level confidence.
4. Run deterministic checks tied to a versioned checklist.
5. Optionally call AI analysis for classification or explanation.
6. Persist findings and their evidence.
7. Display issues to the institution.
8. Institution uploads a new version and reruns checks.
9. On formal submission, officers independently review according to procedure.

If extraction fails or a source cannot be verified, show that state explicitly rather than inferring success.

7. Integration boundaries

- Create interfaces/adapters for external identity, AICTE portal, authorised document verification and messaging.
- For the prototype, use mock adapters and synthetic data unless authorised live access exists.
- Label all mock integrations clearly.
- Never scrape or connect to restricted systems without permission.
- External source verification must record source, timestamp, result and any error when implemented.

8. Failure and recovery

- Failed upload: keep the application state unchanged and offer retry.
- OCR/AI timeout: preserve the uploaded document, mark check as failed/pending, allow retry or manual review.
- Notification failure: record failure and retry; do not mark it sent.
- Concurrent updates: use optimistic concurrency/version checks to prevent overwriting another user's action.
- Invalid workflow action: reject and explain prerequisite/permission.
- Database failure during transition: transaction rollback; no partial transition.
- External integration unavailable: mark source unavailable and route to manual review.

9. Deployment environments

- Local development with synthetic seed data.
- Demo/staging environment with test accounts and no production secrets.
- Production only after security review, approved rules, data governance and authorised integrations.
- Store secrets in environment/secret manager; never commit `.env` secrets.
- Add database migrations, backups and deployment notes.

10. Architecture decision principles

- Prefer simple, testable modules over premature microservices.
- Keep rules configurable and versioned.
- Keep AI optional and replaceable.
- Keep human statutory decisions explicit.
- Keep all privileged operations server-authorised and auditable.